



8기 프로젝트

팀 RAG

프로젝트 소개

2025 국립국어원 AI 말평 경진대회

- 주제

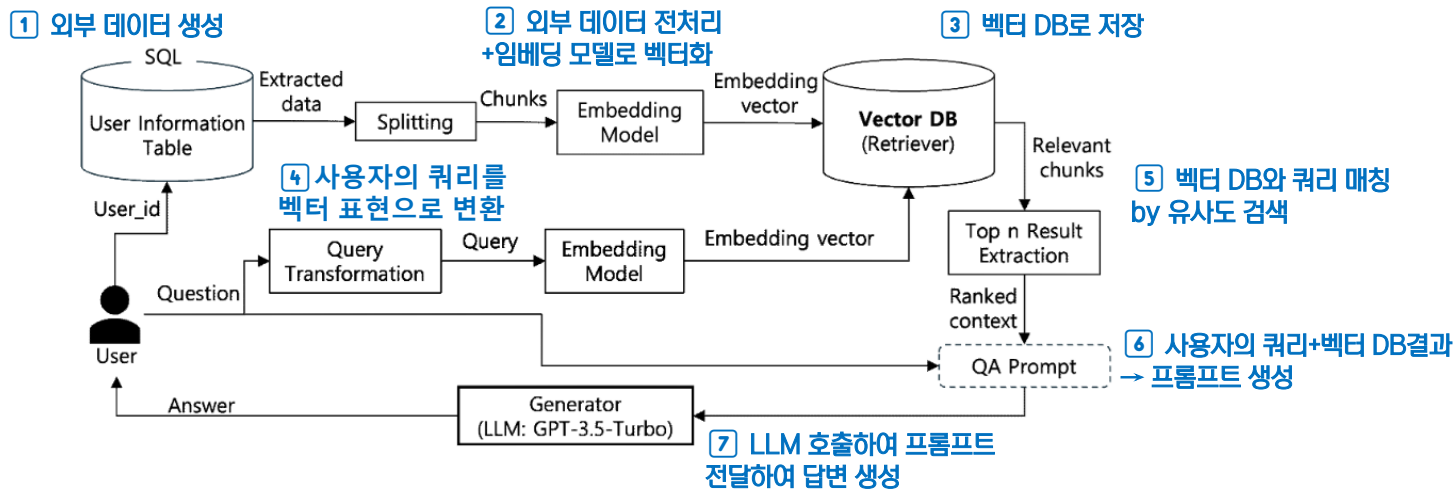
‘한국어 어문 규범 기반 생성(RAG)’ - 한국어 어문 규범 관련 질문에 대하여 주어진 국어 지식을 참조하여 답변을 생성하는 RAG 모델 개발

- 유형

유형	항목	내용
선택형	문제	"가축을 기를 때에는 {먹이량/먹이양}을 조절해 주어야 한다." 가운데 올바른 것을 선택하고, 그 이유를 설명하세요.
	정답	"가축을 기를 때에는 먹이양을 조절해 주어야 한다."가 옳다. 한 음절의 한자어는 앞말이 고유어나 외래어일 때는 독립적인 한 단어로 인식하여 두음 법칙을 적용하고, 앞말이 한자어일 때는 하나의 단어로 인정하지 않아 두음 법칙을 적용하지 않는다. 따라서 한자어 '量'은 앞말이 고유어나 외래어일 때는 '양'이 되고 한자어일 때는 '량'이 된다. '먹이'는 고유어이므로 '먹이양'이 맞는 말이다.
교정형	문제	다음 문장에서 어문 규범에 부합하지 않는 부분을 찾아 고치고, 그렇게 고친 이유를 설명하세요. "어서 꺾차하시길 바래요."
	정답	"어서 꺾차하시길 바라요."가 옳다. 동사 '바라다'에 어미 '-아요'가 결합한 형태이므로 '바라요'로 표기해야 한다. '바램', '바래요', '바랬다'는 모두 비표준형이고 '바람', '바라요', '바랐다'가 표준형이다.



RAG란?



RAG (Retrieval 검색 - Augmented 증강 - Generation 생성)

1. 사용자의 질문을 벡터화하여 대규모 데이터베이스에서 관련 문서를 검색
2. 검색된 문서를 생성 모델에 응답 생성에 필요한 맥락(context)을 추가
3. 검색된 맥락을 LLM에 제공하여 사용자의 질문에 대한 최종 응답을 생성

데이터 전처리

주어진 어문 규정 pdf → jsonl 파일

1. fitz(PyMuPDF) 를 사용하여 PDF 페이지별 텍스트 추출
2. <제목> 내용 구조 패턴을 탐색 → 규칙 단위로 분할
3. 본문 내용 / 예시 구분 및 정제
 - ■● 등 불필요한 특수기호 제거
 - (ㄱ 을 표준어로 삼고, ㄴ 을 버림) 등의 문구가 포함된 경우에는 ㄱ/ㄴ 을 예시와 틀린 예시로 다시 분류
 - 줄바꿈 → 공백 처리

<한글 맞춤법, 표준어 규정 - 표준어 사정 원칙 제10항>

다음 단어는 모음이 단순화한 형태를 표준어로 삼는다.(ㄱ을 표준어로 삼고, ㄴ을 버림.)

- ㄱ: 과팩-하다, -구면, 미루-나무, 미륵, 여느, 온-달, 으레, 케케-묵다, 허우대, 허우적-허우적

- ㄴ: 과팍-하다/과팩-하다, -구면, 미류-나무, 미력, 여늬, 윈-달, 으레, 케케-묵다, 허위대, 허위적-허위적



```
{
  "id": "rule_51",
  "title": "한글 맞춤법, 표준어 규정 - 표준어 사정 원칙 제10항",
  "content": (
    "다음 단어는 모음이 단순화한 형태를 표준어로 삼는다.(ㄱ을 표준어로 삼고, ㄴ을 버림.) "
    "우적 허위적-허위적 "
    "예시: 과팍-하다, -구면, 미루-나무, 미륵, 여느, 온-달, 으레, 케케-묵다, 허우대, 허우적-허 "
    "틀린 예시: 과팍-하다, 과팩-하다, -구면, 미류-나무, 미력, 여늬, 윈-달, 으레, 케케-묵다, 허위대"
  )
}
```

데이터 전처리

주어진 어문 규정 pdf → jsonl 파일

4. 내용 슬라이싱

- 대부분의 모델 (Qwen 등) 에 입력 토큰 수 제한이 있음
- 한 덩어리에 너무 많은 내용이 들어가면 문맥이 흐려질 수 있음

300자 기준으로 나누되,
슬라이딩 윈도우 방식을 사용해 마지막 50 단어를 겹치도록
오버랩 형성

```
{
  "id": "rule_103_1",
  "title": "한글 맞춤법, 표준어 규정 - 한글 맞춤법 제40항",
  "content": "\"\"\"어간의 끝음절 '하'의 'ㅏ'가 줄고 'ㅎ'이 다음 음절의 첫소리와 어울려 거센소리로 될 적에  
[붙임 1] 'ㅎ'이 어간의 끝소리로 굳어진 것은 받침으로 적는다.  
릴지, 아무릴든지, 어떨다, 어떨고, 어떨지, 어떨든지, 이렇다, 이렇고, 이렇지, 이렇든지, 저렇다, 저렇고, 저  
[붙임 2] 어간의 끝음절 '하'가 아주 줄 적에는 준 대로 적는다.  
다, 섭섭하지 않다, 익숙하지 않다 다, 익숙지 않다  
[붙임 3] 다음과 같은 부사는 소리대로 적는다.\"\"\"
}
```

```
{
  "id": "rule_103_2",
  "title": "한글 맞춤법, 표준어 규정 - 한글 맞춤법 제40항",
  "content": "\"\"\"적는다.
[붙임 1] 'ㅎ'이 어간의 끝소리로 굳어진 것은 받침으로 적는다.
릴지, 아무릴든지, 어떨다, 어떨고, 어떨지, 어떨든지, 이렇다, 이렇고, 이렇지, 이렇든지, 저렇다, 저렇고, 저
[붙임 2] 어간의 끝음절 '하'가 아주 줄 적에는 준 대로 적는다.
다, 섭섭하지 않다, 익숙하지 않다 다, 익숙지 않다
[붙임 3] 다음과 같은 부사는 소리대로 적는다.
코 예시:
본말: 간편하게, 연구하도록, 가하다, 다정하다, 정결하다, 혼하다
준말: 간편케, 연구토록, 가타, 다정타, 정결타, 혼타, 앓다, 앓고, 앓지, 앓든지
그렇다, 그렇고, 그렇지, 그렇든지, 아무렇다, 아무렇고, 아무
본말: 거북하지, 생각하건대, 생각하다 못해, 깨끗하지 않다, 넉넉하지 않다, 못하지 않
준말: 거북지, 생각건대, 생각다 못해, 깨끗지 않다, 넉넉지 않다, 못지않다, 섭섭지 않
결단코, 결코, 기필코, 무심코, 아무튼, 요컨대, 정녕코, 필연코, 하마터면, 하여튼, 한사\"\"\"
}
```

그래프 임베딩

1. 그래프 구성

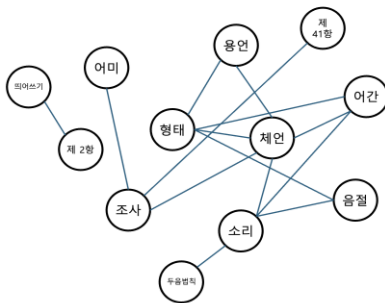
- 노드

- 어문 규범 '조항' 노드
: <> 로 분류

- '핵심 개념' 또는 '규범 용어' 노드
: TF-IDF 기반 키워드 추출, 빈도 분석

- 엣지

- 의미적 유사성 기반 연결



<띄어쓰기 - 한글 맞춤법 제2항>

문장의 각 단어는 띄어 씀을 원칙으로 한다.

<띄어쓰기 - 한글 맞춤법 제41항>

조사는 그 앞말에 붙여 쓴다.

- 꽃이, 꽃마지, 꽃밖에, 꽃에서부터, 꽃으로만, 꽃이나마, 꽃이다, 꽃입니다, 꽃처럼, 어디
까지나, 거기도, 멀리는, 웃고만

+

```
concept_terms = [  
    "소리", "음절", "형태", "제언", "조사", "어간", "어미", "음언", "명사", "부사",  
    "품사", "접미사", "합성어", "파생어", "준말", "고유 명사", "의존 명사",  
    "보조 용언", "한자어", "고유어", "된소리", "구개음화", "두음 법칙",  
    "띄어쓰기", "문장 부호", "축약", "원형"  
]
```



```
{  
    "weight": 0.7592827081680298,  
    "source": "원형",  
    "target": "형태"  
},  
{  
    "weight": 0.7667331695556641,  
    "source": "띄어쓰기 - 한글 맞춤법 제43항",  
    "target": "외래어 표기법 - 외래어 표기법 제3장 제1절 제8항"  
},  
{  
    "weight": 0.76312255859375,  
    "source": "띄어쓰기 - 한글 맞춤법 제43항",  
    "target": "한글 맞춤법, 표준어 규정 - 한글 맞춤법 제11항"  
},  
{  
    "weight": 0.7798619270324707,  
    "source": "띄어쓰기 - 한글 맞춤법 제43항",  
    "target": "한글 맞춤법, 표준어 규정 - 한글 맞춤법 제19항"  
},  
}
```

EWCHA
EURON[illegible]

RAG 파이프라인 설계

```
print("모델 선택")
model_choice = input().strip()
tokenizer, model = load_model(model_choice) 1

print("임베딩 모델 로드")
embedder = SentenceTransformer("jhgan/ko-sroberta-multitask") 2
# embedder = SentenceTransformer("jhgan/ko-sbert-multitask")
# embedder = SentenceTransformer("jhgan/ko-sbert-nli")
# embedder = SentenceTransformer("BM-K/KoSimCSE-roberta")
# embedder = SentenceTransformer("BM-K/KoSimCSE-bert")
# embedder = SentenceTransformer("jhgan/ko-sbert-sts")
# embedder = SentenceTransformer("snunlp/KR-SBERT-V40K-klueNLI-augSTS")

print("규칙 로드 & 인덱싱")
collection, rules = load_and_index_rules_chroma(embedder) 3

print("테스트 데이터 로드")
test_data = load_test_data() 4

print("생성 전략 선택")
strategy = input().strip() 5

print("답변 생성")
results = generate_answers(collection, rules, test_data, tokenizer, model, embedder, strategy=strategy) 6

print("결과 저장")
save_results(results) 7
```

```
def generate_greedy(enc, model, tokenizer, max_new_tokens=256):
    ilen = enc["input_ids"].shape[1]
    out = model.generate(
        **enc,
        max_length=ilen + max_new_tokens,
        eos_token_id=tokenizer.eos_token_id,
        pad_token_id=tokenizer.eos_token_id,
        do_sample=False,
    )
    return tokenizer.decode(out[0][ilen:], skip_special_tokens=True)
```

5-1 greedy

1.LLM 모델 선택(Qwen/llama3)

- 사용자가 사용할 LLM 선택하고, 토큰라이저와 모델을 불러옴
- 답변 생성에 사용됨

2.문장 임베딩 모델 로드

- 문장을 숫자 벡터로 바꿔줄 모델을 불러옴
- 규칙과 질문을 벡터로 변환해서 검색에 사용됨

3.규칙(지식) 인덱싱

- 전처리된 jsonl 파일에서 각 규칙을 읽고 위에서 로드한 embedder를 사용해서 벡터로 변환한뒤 ChromaDB에 저장
- 나중에 질문과 유사한 규칙을 검색하는데 사용

4.질문(테스트 데이터) 로드

5.답변 생성 전략 선택(greedy/top-k 등)

- RAG가 어떤 방식으로 규칙을 활용해 답을 생성할지 선택

```
[ ] def generate_top_k(enc, model, tokenizer, max_new_tokens=256, top_k=3, temperature=0.7):
    ilen = enc["input_ids"].shape[1]
    out = model.generate(
        **enc,
        max_length=ilen + max_new_tokens,
        eos_token_id=tokenizer.eos_token_id,
        pad_token_id=tokenizer.eos_token_id,
        do_sample=True,
        top_k=top_k
    )
    return tokenizer.decode(out[0][ilen:], skip_special_tokens=True)
```

5-2 top-k

RAG 파이프라인 설계

6 프롬프트

```
def generate_answers(collection, rules, test_data, tokenizer, model, embedder, strategy="top-k", top_k=3):
```

```
    results, device = [], model.device
    type_inst = {
        "교정형": """
문장에서 한국어 어문 규범에 맞지 않는 부분을 찾아 올바르게 고치거나,
()에 들어갈 문장 부호를 찾아서,
**수정된 전체 문장**을 작성한 뒤,
"{전체 문장}"가 옳다." 형태로 출력하고 이유를 간단히 설명하시오.
""",
        "선택형": """
질문에 제시된 중괄호({}) 안의 두 단어 중에서 어문 규범에 맞는 단어를 선택해,
**선택한 단어를 포함한 전체 문장**을 작성한 뒤,
"{전체 문장}"가 옳다." 형태로 출력하고 이유를 간단히 설명하시오.
""",
    }

    fmt_map = {
        "교정형": '답: \{"~\}"가 옳다. ...',
        "선택형": '답: \{"~\}"가 옳다. ...',
    }

    for item in test_data:
        qid = item["id"]
        qtype = item["input"]["question_type"]
        question = item["input"]["question"]
        inst = type_inst.get(qtype, "답하시오.")
        fmt = fmt_map.get(qtype, "답: ...")

        qvec = embedder.encode(question).tolist()
        res = collection.query(query_embeddings=qvec, n_results=1)
        retrieved = f"제목: {res['metadata'][0][0]['title']}\n내용: {res[0]['text']}"

        prompt = f"""
다음 질문에 대해 올바른 답을 작성하시오.
- 질문, 규칙, 지침은 출력하지 말고 답변만 출력하시오.
- 답변은 반드시 {fmt} 형식으로 작성하시오.
- 답변은 한 번만 작성하시오. 중복 금지.

[참고 규칙]
{retrieved}

[지침]
{inst}

[질문]
{question}

답:
"""
```

6.질문별로 관련 규칙 검색 + 답변 생성

- 질문 하나하나에 대해 질문을 임베딩으로 바꾸고
- ChromaDB에서 유사한 규칙을 선택한 생성전략으로 검색한 뒤
- 질문 + 규칙을 하나의 프롬프트로 구성해서
- model에 넣고 답변을 생성

7.결과 저장

```
답변 생성
[ID] 750
[Q] 다음 중 어문 규범에 부합하는 것을 선택하고, 그 이유를 설명하세요.
- 다시는 결혼하고 싶지 않아요,
- 다시는 결혼하고 싶지 않아요.
[A] "다시는 결혼하고 싶지 않아요."가 옳다. 문장 끝에 마침표가 들어가야 하므로, 쉼표는 부적절하다.

[ID] 751
[Q] "{에스케이더/에스컬레이터} 손잡이를 잡아야 한다." 가운데 올바른 것을 선택하고, 그 이유를 설명하세요.
[A] "에스케이더 손잡이를 잡아야 한다."가 옳다. '에스케이더'는 국제표준으로 채택된 공식 명칭이며, '에스컬레이터'는 잘못된 발음으로 인한 오류이다.

[ID] 752
[Q] "그는 가끔 {가만히/가만가만히} 생각하는 버릇이 있다." 가운데 올바른 것을 선택하고, 그 이유를 설명하세요.
[A] "그는 가끔 가만히 생각하는 버릇이 있다."가 옳다. '가만히'는 '-히'로 끝나는 단일 어미를 사용하는 형용사로, '가만'이라는 단어에 '-히'를 붙여서 부사어로 쓰는

[ID] 753
[Q] 다음 문장에서 어문 규범에 부합하지 않는 부분을 찾아 고치고, 그 이유를 설명하세요.
"이 글은 사회적 약자들의 목소리에 주목했다는 점에서 의미가 있다."
[A] "이 글은 사회적 약자들의 목소리에 주목했다는 점에서 의미가 있다."가 옳다. '읽기'는 '이익'으로 옳다. '읽기'는 '이익'으로 옳다. '읽기'는 '이익'으로 옳다. '읽기'는 '이익'으로 옳다.
```

QLoRA fine tuning

LoRA(Low-Rank Adaptation)

- 파라미터수가 많은 대규모 언어 모델을 Fine-Tuning하는 메소드로 전체 모델 파라미터를 업데이트하지 않고, 일부 파라미터만 학습시킨다.
- Model weight 를 freeze하고 (LoRA_A X LoRA_B)한 행렬곱을 Transformer Layer 에 있는 Query, Key, Value, Output(=self attention)에 더해준다.

QLoRA

- LoRA를 4-bit 양자화 모델에 결합 -> 메모리와 저장공간 절약 및 성능 동결

출처 : [QLoRA: Efficient Finetuning of Quantized LLMs], [LoRA: Low-Rank Adaptation of Large Language Models]

```
#2. PEFT
lora_config=LoraConfig(
    r=16, #미세 조정 과정에서 학습된 낮은 순위 행렬의 순위 -> r이 낮으면 속도가 빨라지는 대신 성능 저하
    lora_alpha=32, # 학습 속도 또는 학습률 조정, r*2 추천하더라..
    lora_dropout=0.1,
    #target_modules=["q_proj", "v_proj"],
    target_modules=["q_proj","v_proj","gate_proj","up_proj","down_proj"],
    bias="none",
    task_type="CAUSAL_LM",
)
model = prepare_model_for_kbit_training(model)
model=get_peft_model(model,lora_config)
```

QLoRA
4비트 양자화된 모델을 training
+앞서 load_in_4bit=True 설정

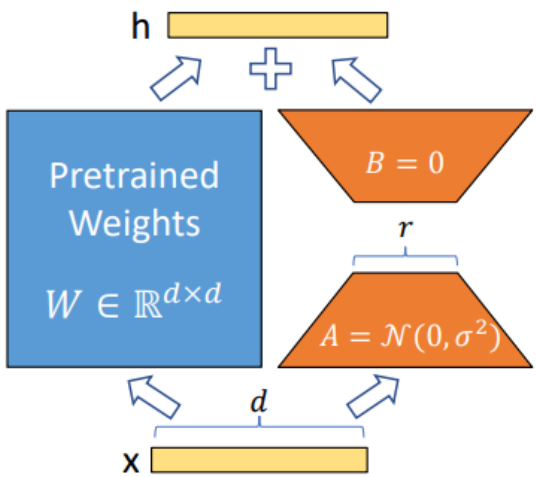


Figure 1: Our reparametrization. We only train *A* and *B*.

LoRAConfig 클래스 정의

- *r* : LoRA 어텐션 차원
- *lora_alpha* : scaling을 위한 파라미터, 보통 *r**2의 정수를 추천
- *target_modules* : adapter를 적용할 Layer들

QLoRA fine tuning

```
# 5. 학습모델
from torch.optim import AdamW
train_param=TrainingArguments(
    output_dir="/content/drive/MyDrive/lor_a_models/llama-ko8b_ver2",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
    save_steps=100,
    logging_steps=10,
    learning_rate=2e-4,
    weight_decay=0.001,
    fp16=False,
    bf16=False,
    gradient_checkpointing=False,
    max_grad_norm=0.3,
    max_steps=-1,
    warmup_ratio=0.03,
    group_by_length=True,
    lr_scheduler_type="constant",
    report_to="none"
)
```

```
optimizer = AdamW(model.parameters(), lr=2e-4)
```

```
trainer=SFTTrainer(
    model=model,
    args=train_param,
    train_dataset=train_dataset,
    optimizers=(optimizer, None) # optimizer 이슈 -> AMP 또는 GradScaler 강제
    #dataset_text_field="text"
    #tokenizer=tokenizer, #더이상 지원 X
    #packing=False #더이상 지원 X
)
```

Training Arguments 정의

: 학습에 적용될 하이퍼파라미터와 로그, 저장 경로 등을 지정하는 클래스

- output_dir : 학습된 모델 체크포인트 저장하는 경로
- per_device_train_batch_size : GPU 1개당 batch크기
- lr_scheduler_type="constant" : learning_rate을 일정하게 유지
- save_steps=100 : 100step마다 체크포인트 저장

LoRA학습을 위한 SFTTrainer 객체

- model : 앞서 불러온 pre-trained된 llm 모델
- args : 위에서 설정한 train arguments
- train_dataset : 학습에 사용될 데이터셋, 여기서는 train.json 파일을 tokenize함
- optimizer : AMP와 GradScaler 관련 충돌 이슈로 AdamW 옵티마이저 사용하여 수동으로 넣어준다.

QLoRA fine tuning

SFTTrainer가 학습 결과를 저장한 LoRA 모델 출력 경로

```
!ls /content/drive/MyDrive/lora_models/llama-ko8b_ver2
```

adapter_config.json	checkpoint-200	tokenizer_config.json
adapter_model.safetensors	checkpoint-234	tokenizer.json
chat_template.jinja	README.md	
checkpoint-100	special_tokens_map.json	

LoRA 어댑터 및 토크나이저 로드

```
# 2. LoRA 어댑터 및 토크나이저 사용
```

```
from peft import PeftModel, PeftConfig
```

```
peft_model_path = "/content/drive/MyDrive/lora_models/llama-ko8b_ver2"
```

Base 모델 위에 LoRA로 학습된 adapter를 연결 → LoRA로 학습된 토크나이저도 로드

```
model = PeftModel.from_pretrained(base_model, peft_model_path)
```

```
tokenizer = AutoTokenizer.from_pretrained(peft_model_path)
```

22	48.6540660	53.9346379 (bleurt, bertscore, ROUGE-1의 평균)	43.3734940	53.9649504	74.5597739	33.2791895	12	28.2203742	46.1997845	10.2409639	46.2947267	69.2653278	23.0392989
25	48.5190175	52.8613281	44.1767068	54.7298919	72.9409636	30.9131288	16	27.5422597	44.6427522	10.4417671	42.9632245	68.1147454	22.8502868
21	48.3397943	53.7077011	42.9718876	54.0761154	74.4125976	32.6343903	11	26.7343892	45.4366498	8.0321285	45.8125077	68.4411950	22.0562468
24	48.2529287	52.9315603	43.5742972	52.8513517	73.7986662	32.1446629	3	24.8369054	45.2561401				
23	47.1004335	51.8313890	42.6947759	51.9380872	73.0683225	30.4909512							21.6758140
26	46.2090773		41.1646586	51.7578033	72.5488845								26.7993530
29	45.7250755	51.8919181	39.5582329	53.9141156	72.2794408	29.4820321							27.3638456
28	44.6383513	49.5176665	39.7590361	48.7421050	71.9416948	27.8691904							25.7518904
27	44.0393142	49.3236083	38.7550201	48.2327844	71.5840561	28.1539812							22.8839256
20	42.0895557	49.8417620	34.3373494	48.9995578	72.3482979	28.1774311							23.2489580
17	39.5800261	47.2323413	31.9277108	46.3731274	70.8633208	24.4605714							19.2060996
19	39.5536527	46.9787914	32.1285141	48.2646641	69.5502421	23.1214679	robertwithQwen3	21.8065264		0.0000000	50.4753666	62.1348696	18.2289220
13	39.4491828	46.7698515	32.1285141	47.6077725	69.5370105	23.1647714	snunlp/chroma	21.6915705	43.1823378	0.2008032	38.7653951	68.3619501	22.4196683
18	39.2120183	46.4963257	31.9277108	46.0723126	69.7800295	23.6366351	jhgan/chroma	21.4244273	42.8488547	0.0000000	38.6469804	68.3211121	21.5784716

성능 테스트 결과

BLEURT (BLEU + RoBERTa + Fine-tuned)

RoBERTa 기반 언어모델을 정답과 유사도 평가 태스크에 맞게 fine-tuning한 후, 사람의 평가 기준에 가깝게 점수를 예측해주는 모델 기반 지표

BERTScore

BERT 모델의 임베딩을 사용하여 출력 문장과 정답 문장의 의미적 유사도를 평가하는 지표

Exact Match

모델의 출력이 정답과 완전히 똑같은 경우를 1로, 다르면 0으로 계산하는 평가 지표

ROUGE-1

출력과 정답 간의 n-gram 겹침 정도를 평가하는 지표
ROUGE-1은 단어(1-gram) 기준으로 겹치는 비율을 측정

팀명	EURON		
모델명	13		
제출 순번	13	제출 일시	2025.07.22. 00:56
평가 점수		39.4491827	
과제명	지표명	평가 점수	
한국어 어문 규범 기반 생성 (RAG)	bleurt	47.6077725	
	bertscore	69.5370105	
	Exact_Match	32.128514	
	ROUGE-1	23.1647714	

팀명	EURON		
모델명	17		
제출 순번	17	제출 일시	2025.07.25. 00:52
평가 점수		39.580026	
과제명	지표명	평가 점수	
한국어 어문 규범 기반 생성 (RAG)	bleurt	46.3731274	
	bertscore	70.8633208	
	Exact_Match	31.9277108	
	ROUGE-1	24.4605756	

팀명	EURON		
모델명	22		
제출 순번	22	제출 일시	2025.07.30. 19:56
평가 점수		48.6540659	
과제명	지표명	평가 점수	
한국어 어문 규범 기반 생성 (RAG)	bleurt	53.9649504	
	bertscore	74.5597739	
	Exact_Match	43.3734939	
	ROUGE-1	33.2791894	

RAG 파이프라인

- Qwen3/Qwen3-8b
- jhgan/ko-sbert-multitask

LoRA 파인 튜닝 1차

- dense search
- beomi/Llama-3-Open-Ko-8B
- BK-K/KoSimCSE-bert

LoRA 파인 튜닝 2차

- hybrid search
- Qwen3/Qwen3-8b
- nlpai-lab/KoE5

THANK YOU

